

Class Notes: Abstract Classes in Java

1. Concrete Classes

Concrete classes are fully implemented classes. This is what we have been working with so far. Since they are complete, we can directly create objects (instances) from them.

Example: Creating an object of a complete **Square** class:

```
Square sq = new Square(5);
```

2. Abstract Classes

Abstract classes are primarily used to support inheritance. They are commonly used to achieve polymorphism and enable generic programming.

To understand abstract classes clearly, it is very important to have a solid understanding of the difference between **reference type** and **actual (object) type**.

3. Abstract Methods

An abstract method:

- 1 Is declared using the **abstract** keyword.
- 2 Does not have a method body.
- 3 Must be implemented (overridden) by concrete subclasses.

Examples of abstract method declarations:

```
public abstract double getArea();  
public abstract double getPerimeter();
```

Important Note: The following is not an abstract method. Although it uses the **abstract** keyword, it incorrectly contains a method body, which is not allowed for abstract methods:

```
public abstract double getArea() { }
```

4. What Makes a Class Abstract?

A class becomes abstract if it contains **at least one abstract method**. Such a class cannot be instantiated directly.

5. Inheritance and Abstract Classes

If a child class inherits from an abstract parent class, then:

- 1 The child class **must override all abstract methods** of the parent class to become a concrete class.
- 2 If it does not override all abstract methods, then the child class itself must be declared abstract.

If a class such as **GeometricObject** is abstract, we cannot create its object directly:

```
GeometricObject go = new GeometricObject(); // Not allowed
```

6. Study Assignment

Extend the implementation provided in the lecture slides by including a concrete **Square** class that inherits from the abstract parent class.

7. Questions & Discussion (Dec 31, 2025)

Question 1: Can a parent class be concrete while the child class is abstract?

Answer: Yes. A concrete class can be extended by an abstract class.

Question 2: Is the following statement valid? Justify your answer:

```
GeometricObject[] objs = new GeometricObject[10];
```

Answer: Yes, this statement is valid. On the left-hand side, the abstract class **GeometricObject** is used as the reference type for the array elements. On the right-hand side, **GeometricObject** is used only to specify the type of objects that the array can store. No actual **GeometricObject** objects are being instantiated here.

8. Constructors in Abstract Classes

It is considered good practice to declare constructors of an abstract class as **protected**.

This ensures that:

- 1 The constructor can be used only by subclasses through inheritance.
- 2 The abstract class cannot be instantiated accidentally by non-subclass classes, especially from a different package.